

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

Работа допущена к защите

Директор ВШПИ

_____ П.Д.Дробинцев

« ____ » _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

работа бакалавра

РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ANDROID-ПРИЛОЖЕНИЯ ДЛЯ СИСТЕМАТИЗАЦИИ ХРАНИМЫХ ПРЕДМЕТОВ

по направлению подготовки (специальности)

09.03.04 Программная инженерия

Направленность (профиль)

**09.03.04_01 Технология разработки и сопровождения качественного
программного продукта**

Выполнил студент гр.

5130904/10102

Г.М.Жуков

Руководитель

доцент, кандидат наук

П.Д.Дробинцев

Консультант

по нормоконтролю

Е.Г.Локшина

Санкт-Петербург
2025

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО

Институт компьютерных наук и кибербезопасности

Высшая школа программной инженерии

УТВЕРЖДАЮ

Директор ВШПИ

_____ П.Д. Дробинцев

«__» _____ 2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Жукову Глебу Михайловичу, группа 5130904/10102

1. Тема работы: Разработка серверной части Android-приложения для систематизации хранимых предметов
2. Срок сдачи студентом законченной работы: 17.05.2025
3. Исходные данные по работе:
 - Документация по языку программирования: Golang
 - Документация по фреймворку: Echo
 - Документация по библиотекам: Golang Goroutines, go-redis, pgx-sql
 - Документация по СУБД: PostgreSQL, Redis
 - Документация по инструментам контейнеризации: Docker, Docker Compose
4. Содержание работы (перечень подлежащих разработке вопросов):
 1. Обзор предметной области
 2. Разработка требований
 3. Выбор технических инструментов
 4. Архитектура приложения

5. Реализация серверной части
6. Тестирование серверной части
7. Развертывание серверной части
5. Перечень графического материала (с указанием обязательных чертежей):
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):
 1. Язык программирования: Golang
 2. Фреймворк: Echo
 3. Библиотеки: pgx-sql, go-redis
 4. Базы данных: PostgreSQL, Redis
 5. Инструменты контейнеризации: Docker, Docker Compose
 6. Облачные сервисы: GitLab CI, Docker HUB
 7. Дополнительные инструменты: Swagger, Git, Postman
7. Консультанты по работе:
8. Дата выдачи задания: 14.04.2025

Руководитель ВКР _____ Дробинцев П. Д.

Задание принял к исполнению 14.04.2025

Студент _____ Жуков Г. М.

РЕФЕРАТ

На 57 с., 13 рисунков, 2 таблицы

КЛЮЧЕВЫЕ СЛОВА: СИСТЕМАТИЗАЦИЯ ПРЕДМЕТОВ, ANDROID, BACKEND, REST API, DOCKER, JWT, REDIS, HTTP, GOLANG, KOTLIN, КЛИЕНТ-СЕРВЕРНАЯ АРХИТЕКТУРА

Тема выпускной квалификационной работы: «Разработка серверной части Android-приложения для систематизации хранимых предметов»

В нынешнее время человек все чаще нуждается в порядке. Частые переезды и смена места жительства стали неотъемлемой частью привычной жизни в обществе новейшего времени. В таком трудоемком, требующем концентрации и ответственности моменте, необходимо эффективно управлять своими личными предметами (под предметами подразумеваются любые материальные объекты, принадлежащие пользователю: одежда, книги, техника, мебель, посуда, инструменты и т. д.). При этом отсутствует гибкий сервис для их организации, с точной системой поиска и фильтрации, а также функцией облачного хранения информации.

Основная цель данного исследования — создание серверного компонента мобильного приложения, предназначенного для эффективной организации, хранения и подсказки к поиску пользовательских предметов.

В работе проводится анализ предметной области, рассматриваются существующие аналоги и их недостатки. Подробно описываются этапы разработки серверной части: проектирование архитектуры, выбор технологического стека, реализация API, обеспечение безопасности данных, тестирование и развертывание. Также рассматриваются вопросы интеграции с клиентским Android-приложением и настройка непрерывной доставки (автоматическое развертывание).

ABSTRACT

57 pages, 13 pictures, 2 tables

KEY WORDS: OBJECT SYSTEMATIZATION, ANDROID, BACKEND, REST API, DOCKER, JWT, REDIS, HTTP, GOLANG, KOTLIN, CLIENT-SERVER ARCHITECTURE

The subject of the graduate qualification work is «Development of the server side of an Android application for the systematization of stored items»

Nowadays, people are increasingly in need of order. Frequent relocation and change of place of residence have become an integral part of everyday life in modern society. In such a time-consuming, demanding concentration and responsibility moment, it is necessary to effectively manage your personal items (items mean any material objects belonging to the user: clothes, books, appliances, furniture, dishes, tools, etc.). At the same time, there is no flexible service for their organization, with an accurate search and filtering system, as well as the function of cloud storage of information.

The main purpose of this study is to create a server component of a mobile application designed to efficiently organize, store, and prompt users to find items.

The paper analyzes the subject area, examines existing analogues and their disadvantages. The stages of server side development are described in detail: architecture design, technology stack selection, API implementation, data security, testing and deployment. Integration with the Android client application and setting up continuous delivery (automatic deployment) are also being considered.

ОГЛАВЛЕНИЕ

Введение	9
Глава 1. Обзор предметной области.....	12
1.1. Первичное техническое задание	12
1.2. Обзор заинтересованных лиц	12
1.3. Обзор аналогов	13
1.3.1. StuffKeeper.....	14
1.3.2. My Boxes.....	14
1.3.3. Home Organizer.....	15
1.3.4. Nai Home Inventory	15
1.3.5. LyfAI.....	16
1.3.6. Сравнительная таблица аналогов	16
1.4. Разработка требований.....	16
1.4.1. Бизнес-требования к приложению	17
1.4.2. Функциональные требования к серверной части	17
1.4.3. Нефункциональные требования к серверной части.....	18
Глава 2. Выбор технических инструментов	20
2.1. Система контроля версий	20
2.2. Язык программирования.....	21
2.2.1. Многопоточный язык программирования - Golang	23
2.2.2. Веб-фреймворк.....	24
2.3. СУБД.....	26
2.3.1. NoSQL подход.....	27

2.3.2. SQL подход	27
Глава 3. Реализация серверной части	30
3.1. Паттерны проектирования.....	31
3.1.1. Паттерн MVC	31
3.2. Архитектура.....	32
3.2.1. Архитектура приложения	33
3.2.2. Архитектура базы данных	35
3.3. Взаимодействие по REST API	37
3.4. Код серверной части	40
3.5. Система безопасности	41
3.5.1. JWT	41
3.5.2. Аутентификация	42
3.6. REST API	43
3.6.1. Описание конечных точек API	43
3.6.2. Структура JSON	44
Глава 4. Анализ результатов	45
4.1. Развертывание серверной части	45
4.1.1. Dockerfile	45
4.1.2. Docker-compose	46
4.1.3. Сервер.....	47
4.1.4. Автоматизация развертывания	47
4.2. Тестирование серверной части.....	50
4.2.1. Используемые технологии	50
4.2.2. Описание методологии тестирования	51
4.3. Автоматизация тестирования	52
4.4. Результаты тестирования	53
4.4.1. Отчет о прохождении тестов	53

4.4.2. Покрытие кода.....	54
Заключение	55
Список используемых источников	56

ВВЕДЕНИЕ

Современные тренды способствуют увеличению потребления человека. Люди покупают вещи не из-за нужды в них, а стараясь следовать определенным тенденциям. Производства стремятся соответствовать запросу и производят огромное количество вещей ежегодно. По данным Федеральной службы государственной статистики, объем промышленного выпуска в январе 2025 года, в том числе сырьевого, увеличился на 2,2% по сравнению с тем же периодом в 2024 году [1].

С целью сделать изготовление предметов более быстрым и экономичным, компании обращаются к философии «Lean». Она направлена на систематизацию процессов производства. Этот современный тренд развивается не только в корпоративной среде, но и в бытовой жизни человека. Люди ежедневно используют такое количество предметов, что нуждаются в их систематизации [2].

Изначально человечество эксплуатировало примитивные методы для данных целей: письменные записи, таблицы или простые программные средства. Только цифровизация меняет восприятие мира. Технологии делают жизнь проще и используемые ранее методы уже не кажутся актуальными. В приведенных условиях главным требованием потребителей становится эффективный учет предметов.

На момент написания данной работы на рынке можно найти лишь единичные мобильные приложения, предлагающие функционал для учета и организации предметов. Они имеют различные вариации и практическую пользу: от систематизации бытовых предметов по виртуальным коробкам до базового управления небольшими хранилищами. Существуют и мощные корпоративные решения для управления складами, но они либо недоступны для свободного скачивания, либо требуют платной подписки и интеграции в бизнес-процессы, поэтому в данном исследовании рассматриваться не будут. При этом приложе-

ния, которые доступны для свободного скачивания ограничены в функциональности: в них отсутствует удобная система сортировки и поиска, синхронизация между устройствами и поддержка облачных технологий, а также имеются слабые возможности для работы с вложенными структурами предметов. Все эти ограничения значительно снижают эффективность таких приложений, требуя разработки более прогрессивных решений, которые способны удовлетворить такие запросы потребителя как гибкость, безопасность и удобстве работы с данными.

Систематизация и учет предметов предполагает создание универсальной системы, которая могла бы работать с различными типами данных и удовлетворять требованиям потребителя в организации, фильтрации и синхронизации. На основе этих принципов предполагается разработка гибкой архитектуры, которая будет включать как клиентскую, так и серверную части, обеспечивающие надежную работу приложения и взаимодействие между различными устройствами пользователя. Серверная часть приложения будет отвечать за обработку данных и их хранение в базе данных.

Таким образом, создание серверного компонента приложения, предназначенного для организации хранимых предметов, является важной задачей, особенно учитывая растущий объем данных и спрос на адаптируемые высокопроизводительные решения для управления. В рамках дипломной работы будет описан комплексный дизайн архитектуры приложения, выбор соответствующих технологий и инструментов, а также реализация основных функциональных возможностей для обеспечения удобного и безопасного обращения с хранимыми предметами.

Создание данного приложения будет не только улучшать процесс учета и управления личными предметами, но и сделает процесс работы с данными более удобным для пользователя и эффективным для его работы.

Итог работы – создание серверной части мобильного приложения для систематизации предметов и ее интеграции с клиентской частью.

Далее представлены задачи, которые были решены во время работы над серверной частью мобильного приложения:

1. Анализ области управления предметами и их организации;
2. Определение требований к приложению;
3. Проектирование архитектуры приложения;
4. Разработка реализации на стороне сервера;
5. Реализация тестового покрытия для кодовой базы;
6. Настройка системы автоматического серверного развертывания;

ГЛАВА 1. ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

В данной главе рассматриваются особенности предметной области, проводится сравнительный анализ конкурентов и разрабатываются требования к приложению. Таким образом, в этом исследовании рассматриваются ключевые аспекты и определяются лучшие практики для последующей реализации программного обеспечения. Результаты создадут базовую основу для будущего анализа и определяют основные функции приложения.

1.1. Первичное техническое задание

В рамках командной работы требуется реализовать Android-приложение для учета и систематизации личных предметов. Актуальность данного проекта исходит из многочисленных запросов пользователей, постоянно сталкивающихся с нуждой в приложении для эффективного хранения и поиска предметов в повседневной жизни — от коллекционных артефактов до бытовых принадлежностей и инструментов.

В этом проекте реализовано серверное решение с кроссплатформенной совместимостью, способное интегрироваться с любым клиентским приложением, поддерживающим указанный протокол связи, независимо от целевого сообщества пользователей.

1.2. Обзор заинтересованных лиц

Основная целевая аудитория разрабатываемого приложения разнообразна:

- Частные пользователи, чья потребительская нужда состоит в стремлении к эффективной организации хранения личных предметов: бытовые и коллекционные предметы, одежда, техника и т. д.
- Групповые сообщества, чья потребительская нужда состоит в стремлении к эффективной организации в рабочем пространстве, особен-

но в креативных сферах.

Приложение ориентировано на широкий круг пользователей, стремясь закрыть потребность в комфортном и простом способе упорядочивании предметов.

Основная задача разработки приложения – создать интерфейс, который не вызывал бы трудностей при использовании как технически продвинутым пользователям, так и тем, кто не обладает такими знаниями. В связи с этим особое внимание уделяется удобству пользовательского взаимодействия и минимизации порога входа.

Ниже приведены следующие элементы, которые важны и необходимы при проектировании и разработке данного приложения:

- Настраиваемые схемы хранения с упрощенными интерфейсами управления;
- Сквозное шифрование и контроль доступа для облачных развертываний;
- Модульная конструкция серверной части, обеспечивающая независимую от клиента расширяемость;

1.3. Обзор аналогов

Пользователи новейшего времени, сталкиваясь с проблемой, часто обращаются к цифровым решениям. Одной из таких задач, вставшей перед современным человеком, является система хранения предметов (бытовых, коллекционных и т. д.). При этом цифровой рынок не стремится решать эту проблему. Проведя анализ рынка существующих приложений по данной тематике, оказалось, что большая часть существующих приложений имеет существенные ограничения, влияющие на их эффективность в негативную сторону. В данном разделе рассматриваются сервисы для учета предметов, их функциональные возможности, ключевые недостатки и проблемы, с которыми сталкиваются пользователи.

1.3.1. StuffKeeper

Данное приложение является инструментом для управления личными предметами [3]. В доступе пользователя: создание виртуальных хранилищ, прикрепление изображений предметов и трекер-слежения их местоположения. Последнее является основным преимуществом «StuffKeeper», однако функциональная часть имеет свои ограничения:

- Отсутствие возможности редактирования местоположения предмета после его внесения в приложение;
- Отсутствие поддержки вложенных структур (например, ”шкаф → полка → коробка”);
- Отсутствие фильтрации по дополнительным параметрам (поиска осуществляет только по названию);

Более того, стоит подчеркнуть, что синхронизации данных между несколькими устройствами нестабильна с учетом наличия облачного хранилища.

1.3.2. My Boxes

Данное приложение в том числе ориентировано на организацию предметов [4]. Пользователь может положить их в виртуальную коробку, маркировать ее, описывать содержимое и создавать простые категории. Однако в «My Boxes» не предусмотрена гибкость:

- Отсутствует поддержка многоуровневой иерархии, например коробки внутри других коробок;
- Отсутствует система синхронизация между устройствами;
- Отсутствует удобная фильтрация: поиск не осуществляется по тегам или атрибутам предметов;

1.3.3. Home Organizer

Основное предназначение данного приложения – организация бытовых предметов [5]. Пользователь может использовать такие возможности как: группирование предметов по спискам, присваивание им категорий и добавление минимального описания. Однако функционал приложения ограничен:

- Отсутствует удобная фильтрация поиска: по тегам и ключевым словам;
- Отсутствует трекер-геолокации: поиск предметов в реальном пространстве невозможен;
- Отсутствует возможность создания пользовательских полей для более конкретного описания атрибутов («год выпуска», «срок годности» и т. д);

Использование приложения предназначено лишь для простого учета, и особо затруднительно для распределения сложных данных.

1.3.4. Nai Home Inventory

Данное приложение обладает простейшими и минимальными функциями для организации различных предметов [6]. Возможности пользователя – добавление предметов, назначение им категорий и сканирование QR-кодов. Основными серьезными ограничениями являются следующие параметры:

- Приложение позволяет сканировать лишь существующие QR-коды;
- Отсутствует возможность расширенной фильтрации по созданным пользователем тегам;
- Отсутствует система синхронизация между устройствами;

Приложение «Naic Home Inventory» не соответствует пользовательскому запросу на гибкие и персонализированные задачи.

1.3.5. LyfAI

Данное приложение обладает визуализацией данных [7]. В возможностях пользователя: добавлять и просматривать предметы в виде сетки или списка. При этом данная система имеет строгую структуру, из-за чего функционал программы сильно ограничен:

- Отсутствие возможности настраивания иерархии хранилищ, например генерирование подкатегорий;
- Отсутствует система синхронизация между устройствами;
- Отсутствует система распознавания QR-кодов или определения местоположения предмета;

Приложение «LyfAI» применимо лишь для примитивных задач, с отсутствием возможности для координирования сложных данных.

1.3.6. Сравнительная таблица аналогов

Таблица 1 Сравнительный анализ функциональности

Инструмент	Редактируемые поля	QR-коды	Иерархия предметов	Поиск и сортировка	Облачное хранилище	Геолокация
StuffKeeper	-	-	-	+	+	-
My Boxes	-	-	+	+	+	-
Home Organizer	-	-	-	-	-	-
Naic Home Inventory	-	+	-	+	-	-
LyfAI	-	-	-	-	-	-

1.4. Разработка требований

На начальном этапе реализации разрабатываемого приложения первоначальным действием станет формализация требований: определение его архитектуры, функциональности и основных сценариев использования.

В рамках данного проекта предполагается создание клиент ориентированного сервера приложения для телефона. Его главное предназначение: учет, хранение и систематизация личных предметов. В качестве основной функций,

которые представляются пользователю, можно выделить: создание предметов, прикрепления к ним изображений и расширенного описания, возможность задавать характеристики и удобная фильтрация при поиске по различным параметрам, сканирование и генерация QR-кодов для идентификации предметов.

Формат мобильного приложения коррелирует с актуальностью этого цифрового носителя, как центрального проводника в технологичный мир. Так как в современном мире человек почти постоянно находится в непосредственной близости с телефоном, то это позволит пользователям приложения вносить изменения, проверять местоположение и просматривать коллекцию в любое время и в любом месте.

1.4.1. Бизнес-требования к приложению

- Формат проекта: серверная часть приложения, поддерживающая масштабирование для веб-интерфейса или других клиентских решений;
- Устройства: клиентский компонент изначально разрабатывался для устройств Android, но серверная часть должна быть универсальной, способной взаимодействовать со всеми типами клиентов;
- Целевая аудитория: частные пользователи и компании с запросом о систематизации предметов (инвентаря, офисных предметов, коллекций, инструментов и других);
- Практическая ценность: сервисное приложение направлено на эффективное хранение и учет предметов, а также их быстрый поиск по тегам и прикрепление изображений;

1.4.2. Функциональные требования к серверной части

Функциональные возможности пользователей определяются их ролью в системе. При этом каждая последующая роль включает в себя весь функционал предыдущей. Таким образом, зарегистрированный пользователь имеет доступ

к функциям гостя, а администратор — ко всем функциям, включая управление системой.

Неавторизованный пользователь:

- регистрация и авторизация;
- получение информации о приложении;
- демонстрационный просмотр предметов (ограниченный доступ);

Авторизованный пользователь:

- работа с личными предметами
 - добавление нового предмета;
 - редактирование и удаление своих предметов;
 - добавление изображений и метаданных (дата, местоположение, статус и др.);
 - получение списка предметов;
 - поиск предметов по названию, описанию и тегам;
- личный кабинет
 - просмотр и редактирование личной информации (имя, email);
 - обновление пароля с подтверждением ввода старого пароля;
- дополнительные функции
 - экспорт данных (в формате JSON или CSV);

1.4.3. Нефункциональные требования к серверной части

Безопасность:

- обеспечение безопасной аутентификации пользователей;
- защита персональных данных и учетных записей от несанкционированного доступа;
- использование шифрования для хранения чувствительной информации (паролей);

Управляемость:

- система должна поддерживать логирование действий пользователей и системных событий;
- обеспечение возможности мониторинга состояния сервиса (статистика запросов, ошибок, загрузки и т.д.);

Контроль информации:

- предотвращение SQL-инъекций при вводе данных;

Масштабируемость:

- архитектура системы должна предусматривать возможность горизонтального масштабирования без существенного снижения производительности при увеличении числа пользователей или объема данных;

ГЛАВА 2. ВЫБОР ТЕХНИЧЕСКИХ ИНСТРУМЕНТОВ

Для достижения поставленной цели данной дипломной работы необходимо подобрать корректные технические инструменты и выбрать подходы к построению архитектуры приложения. Требуется проверить совместимость подходов и программных инструментов, проанализировать для дальнейшего использования готовые библиотеки и фреймворки, которые отвечают требованиям продукта.

2.1. Система контроля версий

Описание выбора технических инструментов будет корректно начать с упоминания применимой системы контроля версий кода. Ведь это инструмент, без которого разработчик не сможет справиться. Он помогает управлять изменяющимися данным в исходном коде и сохраняет промежуточные версии проекта. Представленная система помогает процессу разработки в команде быть более безопасным и гибким:

- Git – самая известная система контроля. По сравнению с альтернативными вариантами контроля версий, он демонстрирует высокую производительность благодаря оптимизации процедур фиксации коммитов, создания веток, слияния и сравнения предыдущих версий. Также эта система контроля гибка: она поддерживает нелинейный цикл разработки, что является ценным аспектом командной работы;
- GitLab и GitHub – самые часто используемые веб-сервиса для хостинга проектов с использованием этой версии системы контроля;
- GitLab CI/CD – встроенные в GitLab инструменты для автоматизации процессов тестирования, сборки приложения и развертывания;
- GitLab – платформа для координации кода, разрабатываемого в рамках данной работы веб-приложения и совместной разработки. Также

группа проектов разделяется еще на два подпроекта: один - ответственный за серверную часть, второй – за клиентскую;

2.2. Язык программирования

На сегодняшний день существует множество языков высокого уровня, которые предоставляют современные возможности для написания различных программных продуктов и сервисов — десктоп-приложений, веб-приложений, мобильных приложений и т. д.

В предыдущем разделе были определены требования к приложению, следовательно, далее необходимо проанализировать текущее состояние сферы языков программирования для выбора наилучшего варианта. Однако в данном случае выбор языка программирования основывается не на рейтинге популярности, а на технических характеристиках, таких как производительность, простота параллельного программирования, легкость масштабирования и стабильность кода при долгосрочной эксплуатации.

Среди существующих языков в качестве основного был выбран Go (или Golang) — компилируемый язык программирования, разработанный компанией Google и предназначенный для создания эффективных и масштабируемых серверных решений. Go предоставляет встроенные средства для конкурентного программирования (goroutines), что делает его особенно удобным для построения высоконагруженных распределенных систем.

На рис. 1 представлена визуализация результатов сравнительного тестирования производительности языков программирования, проведенного в рамках проекта The Computer Language Benchmarks Game [8].

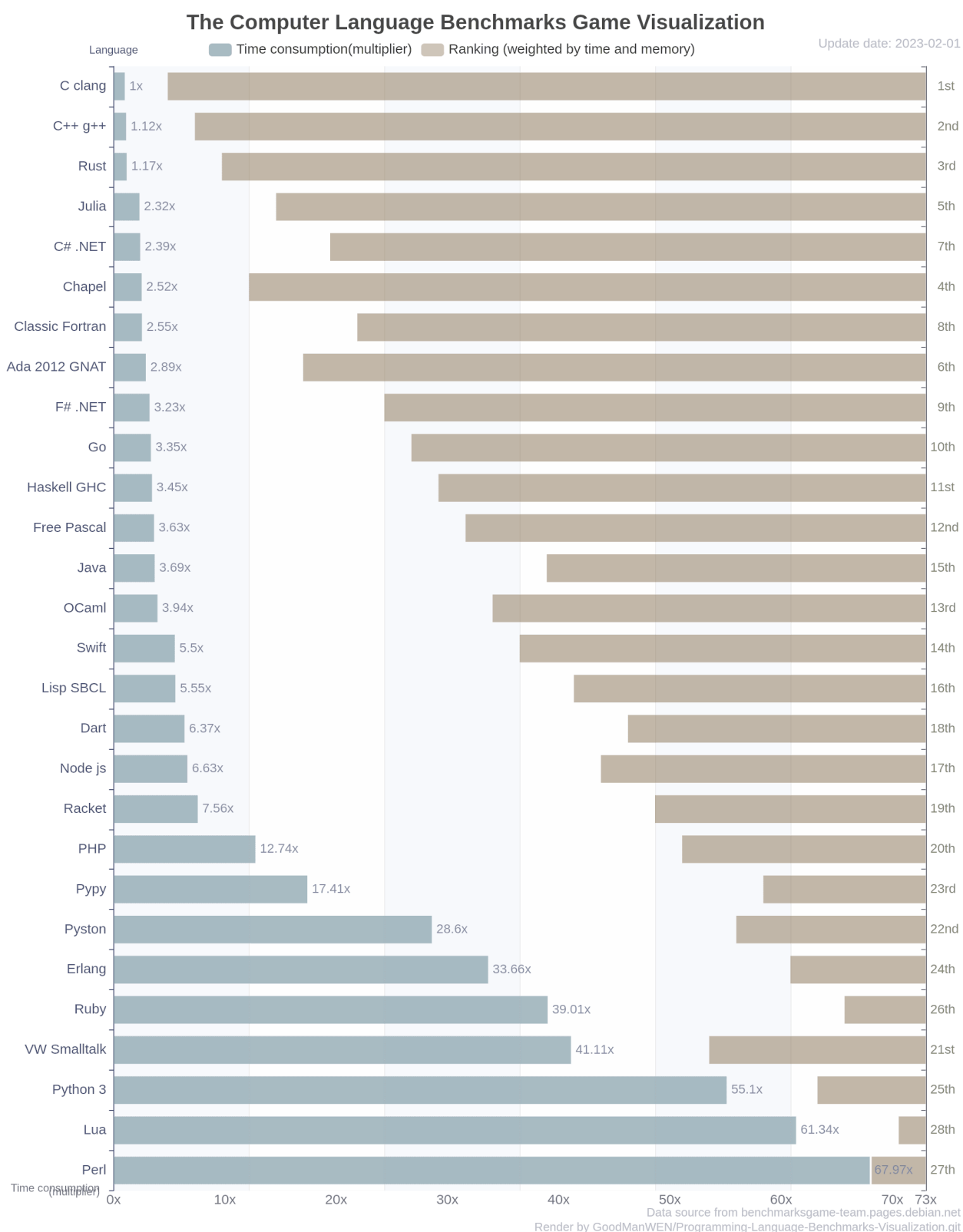


Рис. 1. Визуализация тестов производительности языков программирования

По данным тестирования, язык Go занимает десятое место, по совокупной оценке, времени выполнения и потребления памяти. Этот факт демонстрирует его высокую эффективность и подтверждает пригодность для разработки

производственных серверных решений по сравнению с языками программирования Java и Python, затрачивающих больше времени на выполнение задач.

2.2.1. Многопоточный язык программирования - Golang

Go является строго типизированным языком программирования. Сравнивая его с прямым конкурентом – C++, нельзя не отметить простоту синтаксиса у языка. То есть при написании кода на Go облегчается создание программного продукта. В данном языке содержатся средства поддержки параллелизма, как goroutines и каналы. Именно они помогают реализовывать высоконагруженные распределенные системы и реализовывать микро-сервисную архитектуру.

Одно из главных преимуществ языка Golang: обширная библиотека для разработки полноценного веб-сервиса без сторонних ресурсов. Это облегчает разработку и повышает ее надежность.

Кроме того, именно этот язык программирования обладает высокой скоростью компиляции. Данное преимущество позволяет сократить этапы цикла разработки и мгновенно тестировать внесенные изменения.

Более того, он обеспечивает гибкость при развертывании серверной части приложения, принадлежащей к любой операционной системе. Это возможно благодаря простой сборке кроссплатформенных бинарных файлов и переносимости.

По сравнению с остальными языками программирования, Golang демонстрирует эффективные результаты в масштабных и многопоточных задачах [9]. Это делает его особенно подходящим для задач, где важна стабильная и быстрая обработка большого количества запросов.

На момент написания приложения актуальной и стабильной версией языка программирования Go является Go 1.22. В данной версии присутствует весь необходимый инструментарий для реализации архитектуры программного

продукта и эффективного взаимодействия между компонентами системы.

2.2.2. Веб-фреймворк

Выбор веб-фреймворка играет ключевую роль в построении архитектуры серверной части приложения. От него зависит не только удобство разработки и поддержка кода, но и производительность конечного продукта, его масштабируемость и надежность при длительной эксплуатации.

Среди множества веб-фреймворков, разработанных для языка Go, можно выделить несколько наиболее популярных и активно применяемых в разработке на Go. Для объективной оценки были рассмотрены четыре основные характеристики:

- Производительность, то есть скорость обработки HTTP-запросов, что напрямую влияет на отзывчивость и быстродействие системы;
- Функциональность, под которой подразумевается наличие встроенных инструментов и поддержка различных сценариев разработки (middleware, маршрутизация, работа с шаблонами, валидация и др.);
- Масштабируемость, отражающая, насколько удобно на базе выбранного фреймворка развивать систему и адаптировать ее под высокие нагрузки;
- Активность сообщества, которая позволяет судить о зрелости проекта, наличии документации, примеров, сторонних библиотек и регулярных обновлений;

Ниже представлена сравнительная таблице 2, в которой приведены характеристики наиболее распространенных фреймворков Go, основываясь на их реальных показателях в практическом применении [10]:

Таблица 2. Сравнительный анализ фреймворков

Фреймворк	Производительность	Функциональность	Масштабируемость	Сообщество
Gin	Очень высокая	Богатая	Хорошая	Крупное
Echo	Высокая	Умеренная	Хорошая	Быстрорастущее
Beego	Высокая	Богатая	Средняя	Крупное
Fiber	Очень высокая	Умеренная	Хорошая	Растущее
Revel	Высокая	Богатая	Средняя	Среднее
Go Kit	Средняя	Узко-специализированная	Отличная	Среднее
Buffalo	Средняя	Богатая	Хорошая	Небольшое
FastHTTP	Самая высокая	Ограниченная	Отличная	Крупное

Анализируя данные таблицы, можно отметить, что такие фреймворки, как Gin, Fiber и FastHTTP, демонстрируют очень высокие показатели производительности, что делает их привлекательными для проектов и критичных к времени отклика. Однако не все из них предоставляют достаточный уровень абстракции и функциональности ”из коробки”, что может затруднить реализацию более комплексной бизнес-логики и увеличивать время разработки.

С другой стороны, фреймворки вроде Beego или Revel предлагают более богатый инструментарий, однако они менее гибкие в вопросах масштабируемости и не так активно развиваются.

Учитывая всё вышесказанное, выбор в рамках данной работы был сделан в пользу Echo — легковесного, но в то же время производительного веб-фреймворка, обладающего сбалансированным набором функций. Его отличие в структурированной архитектуре с поддержкой middleware, четкой документации и способностью простой настройки маршрутизации. Именно Echo позволяет быстро обрабатывать запросы, сохраняя отсутствие сложности в использовании. Кроме того, он достаточно гибкий для использования его в задачах с требованием к масштабируемым системам.

Этот веб-фреймворк постоянно развивается, что позволяет строить гипотезу о долговременности использования с его помощью приложений. В итоге он представляет из себя идеальный баланс по наивысшей

производительности, удобству и защищенности, в отличие от других похожих вариантов и становится приемлемым решением для задачи в данной дипломной работе.

2.3. СУБД

База данных – это инструмент хранения информации. Она позволяет эффективно реализовывать поиск, хранение и обработку данных [11].

Система управления базами данных (СУБД) — это специализированное программное обеспечение, предназначенное для создания, сопровождения и взаимодействия с базами данных. Она обеспечивает возможность выполнения различных операций с данными: их добавление, изменение, удаление, выборку, а также управление доступом. Основные функции СУБД включают в себя обеспечение надежности хранения, поддержание целостности информации, реализацию механизмов безопасности и распределение прав пользователей [12].

Базы данных разделяются на два вида:

- Реляционные базы данных (SQL) — реляционные базы данных, использующие четкую схему с таблицами. Они поддерживают язык структурированных запросов для взаимодействия с данными;
- Нереляционные базы данных (NoSQL) — нереляционные базы данных, созданные для случаев, требующих высокой гибкости хранения. Кроме того, их также стоит использовать для работы с не четко структурированной информацией;

Необходимость использования одной из баз данных зависит от архитектурных особенностей разрабатываемой системы, а также объема и характера обрабатываемых данных, требований к масштабируемости, безопасности их хранения и скорости доступа к ним.

2.3.1. NoSQL подход

NoSQL (Not Only SQL) – подход к хранению и обработке данных. Он отличается от классической реляционной модели. Главной задачей, стоящей перед его разработчиками, стала реализация подхода для прогрессивных веб-приложений и распределенных систем. NoSQL может работать с высокой производительностью над масштабируемыми задачами с гибкими структурными данными.

Его главная особенность, отличающая его от других программ – отказ от устойчивой схемы данных и хранение абсолютно разных типов структур, например, от ключ-значения до документов.

Данные особенности дают возможность быстрой адаптации к преобразованиям требований и создавать разбиение системы на более мелкие структурные компоненты.

2.3.2. SQL подход

SQL – подход для управления базами данных. Он достаточно давно используется в разработке. Его подход заключается в хранение данных с четко структурированными и классифицированными таблицами. Кроме того, этот язык программирования используется для получения и удаления данных, а также их модификации и добавления из базы. В различных системах управления применяются измененные формы этого языка, соответствующие стандарту, обновляющимся со временем.

Ниже приведены положительные стороны SQL-подхода:

- Четкая структура данных, состоящая из строгих схем таблиц. Она обеспечивает ясность и целостность составляющей хранения информации;
- Поддержка трудных запросов (в которые входят массивный объем данных и обладающих не простой системой выборки), которые дан-

ный язык программирования способен быстро, четко и эффективно обрабатывать;

- Транзакционность (ACID), которая отвечает свойствам атомарности и согласованности современных систем. Более того, она обладает изолированностью и долговечностью, что автоматически гарантирует безопасное и надежное выполнение задачи;
- Большой опыт использования SQL-системы в целях программирования. Огромное преимущество в большом выборе инструментария и широком круге поддерживаемых сообществ;

Недостатки:

- Строгая схема, с трудом позволяющая внести изменения в структуры таблицы, особенно явно с такой проблемой можно столкнуться на поздних этапах разработки;
- Ограниченные возможности масштабируемости, особенно на несколько серверов. Таким образом SQL-система может не всегда использоваться в распределенных структурах;
- Менее эффективная обработка неструктурированных данных (например, текст, JSON и другие вариации информации) по сравнению с NoSQL-подходом;
- Во время одновременных операций записи и чтения SQL-системы снижают свою производительность;

Ниже представлены основные системы управления базами данных на выбор [13]:

- MySQL является наиболее широко используемой среди основных серверных баз данных. Она наиболее удобна для пользователя, имеет обширную онлайн-документацию и ресурсы. Хотя она не полностью соответствует стандартам SQL, но обеспечивает надежную функциональность. Приложения взаимодействуют с базой данных через специальный процесс;

- PostgreSQL является наиболее технически-продвинутой СУБД, в которой приоритет отдается полному соответствию стандартам SQL и расширяемости. В отличие от других баз данных, она строго соответствует стандартам ANSI/SQL, обеспечивая надежность и расширенную функциональность;
- SQLite является встроенной базой данных, которая работает в самом приложении. Как файловая система баз данных, она предлагает мощный, оптимизированный набор инструментов для обработки данных, что делает ее идеальной для сценариев, в которых серверные базы данных были бы излишними. В отличие от традиционных баз данных, использующих промежуточные интерфейсы, SQLite обеспечивает прямой функциональный доступ к файлам данных (например, к собственным файлам базы данных). Такой подход устраняет накладные расходы, что приводит к ускорению и повышению эффективности операций;

После анализа представленных систем управления базами данных было принято решение в пользу использования PostgreSQL как основной СУБД для разработки программного продукта. Этот выбор обусловлен рядом технических и архитектурных преимуществ, которые делают PostgreSQL наиболее подходящим решением для построения надежной и расширяемой системы.

В первую очередь, PostgreSQL — это полностью реляционная объектно-ориентированная СУБД, которая поддерживает строгую типизацию данных и предоставляет расширенные возможности работы со сложными структурами. Благодаря поддержке языка SQL в полном соответствии со стандартами ANSI/ISO, PostgreSQL обеспечивает высокую степень надежности при выполнении транзакций, что критически важно для обеспечения целостности данных в распределенных и многопользовательских системах.

Особое внимание заслуживает поддержка ACID-свойств (атомарность, согласованность, изолированность, долговечность), что гарантирует коррект-

ность всех операций, даже в случае сбоев или параллельной работы с базой. Кроме того, PostgreSQL предоставляет развитый механизм блокировок, журнал транзакций (Write-Ahead Logging), триггеры, хранимые процедуры и возможности написания собственных расширений, что делает систему гибкой и легко настраиваемой под специфические требования приложения.

С точки зрения производительности, PostgreSQL хорошо масштабируется как вертикально (за счет настройки и оптимизации под конкретное «железо»), так и горизонтально (благодаря встроенным средствам репликации, таким как streaming replication и logical replication). Это делает его подходящим решением как для небольших приложений, так и для высоконагруженных систем, обрабатывающих большие объемы данных.

Немаловажным фактором является и наличие большого количества библиотек, ORM-фреймворков и инструментов для интеграции PostgreSQL с различными языками программирования, включая Go, который был выбран в качестве основного языка разработки в рамках данной дипломной работы.

Таким образом, с учетом всех перечисленных факторов, можно сделать вывод, что использование PostgreSQL позволит обеспечить надежность, гибкость, производительность и масштабируемость системы на всех этапах ее жизненного цикла.

ГЛАВА 3. РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ

Серверное приложение разработано с использованием Go (Golang) 1.22, скомпилированного многопоточного языка программирования, использующего фреймворк Echo для эффективной маршрутизации и поддержки промежуточного программного обеспечения. Для сохранения данных система интегрирует PostgreSQL в качестве основной реляционной базы данных и Redis для высокопроизводительного кэширования. Взаимодействие клиент-сервер осуществляется исключительно с помощью HTTP-запросов.

3.1. Паттерны проектирования

Шаблоны проектирования представляют собой многократно используемые архитектурные решения общих задач разработки программного обеспечения в конкретных контекстах [14]. При структурировании серверной архитектуры важно оценить основные шаблоны проектирования веб-сервисов, чтобы обеспечить оптимальное и масштабируемое решение.

3.1.1. Паттерн MVC

Model-View-Controller(MVC) – это широко используемый архитектурный паттерн, разделяющий приложений на три компонента. Эти компоненты называются модель, представление и контроллер.

Слой модели предоставляет данные и содержит бизнес-логику приложения. Он инкапсулирует состояние приложения и предоставляет методы для доступа и изменения этого состояния. Модель никак не зависит от пользовательского интерфейса и не знает, в каком виде отображаются данные или как пользователь с ними взаимодействует.

Следующий слой –это представление. В контексте REST API данный слой отвечает за данные, которые будут отправлены клиенту. В нашем случае это заключается в преобразовании Golang объектов в JSON объекты, которые затем отправляются на клиент в составе HTTP ответов.

И последний слой –контроллер. Он действует как посредник между моделью и представлением. Контроллер занимается обработкой действий пользователя, интерпретирует их и передает модели. То есть контроллер определяет, как приложение будет реагировать на действия пользователя.

Рассматриваемый паттерн проектирования MVC упрощает разработку, поддержку, тестирование приложений и предоставляет возможность повторного использования кода, так как разделяет приложение на логические части. Разделив код приложения на три отдельных компонента – модель,

представление и контроллер – разработчики могут работать над каждым компонентом независимо друг от друга, что снижает риск возникновения ошибок и минимизирует влияние изменений на общую кодовую базу.

Еще одно достоинство паттерна MVC – это поддержка нескольких пользовательских интерфейсов. Так как компонент представления отвечает за отображение пользовательского интерфейса, появляется возможность создавать и интегрировать различные представления для одних и тех же компонентов модели и контроллера. Такая гибкость позволяет отображать данные приложения в различных интерфейсах, например в мобильном или веб-интерфейсе, не изменяя при этом основную логику приложения. Схема, визуализирующая паттерн MVC, приведена на рис. 2.

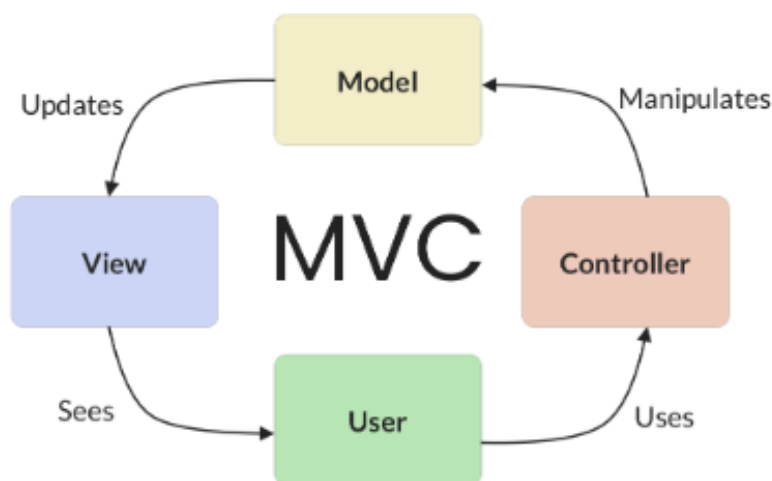


Рис. 2. Шаблон MVC

3.2. Архитектура

Фундаментальный аспект при создании приложения – его архитектура. Если ее точно спроектировать, то появляются возможности: расширения приложения, внесения изменений, добавления новых функций и обеспечения высокой производительности. Это требует больших затрат ресурсов времени и сил на начальном этапе разработки, но при этом является вкладом в будущее.

Архитектура - это база приложения. Корректное проектирование – особый раздел серверной архитектуры, т. к. огромное количество функций (масштабируемость, производительность и удобство в обслуживании зависит от нее).

3.2.1. Архитектура приложения

Приложение для систематизации личных предметов включает в себя серверную и клиентскую части. В данной выпускной квалификационной работе рассматривается разработка серверной части, но для представления всей системы необходимо изучить общую архитектуру приложения, расположенную на рис. 3.

Глобально приложение состоит из трех частей, работающих независимо:

- Клиента, представляющего собой мобильное приложение на ОС Android;
- Сервера, представляющего собой бэкенд, написанный на фреймворке Echo;
- Баз данных - PostgreSQL и Redis;

Пользователь взаимодействует с клиентским приложением, имея дело с UI, написанным с помощью библиотеки Jetpack Compose. Элементы интерфейса вызывают методы соответствующих им ViewModel – классов, отвечающих за логику UI и иногда содержащих в себе небольшое количество бизнес-логики. Таким образом, UI несет ответственность только за отображение элементов и навигацию между ними.

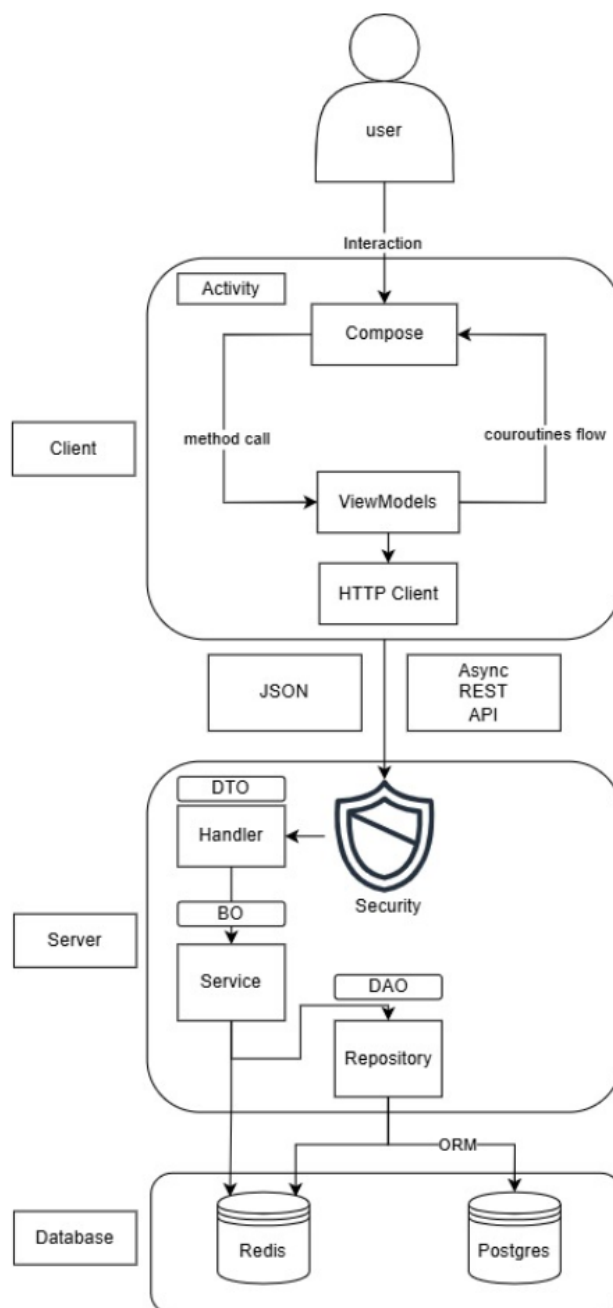


Рис. 3. Общая архитектура приложения

ViewModel, в свою очередь, обращается к REST API сервера при помощи HTTP клиента. Обращение происходит с использованием корутин, соответственно, эти вызовы являются асинхронными и не блокируют поток, что исключает моменты заморозки интерфейса в ожидании ответа сервера. Получив ответ, UI поглощает поток состояний и реактивно отрисовывает у себя актуальное состояние интерфейса.

HTTP запрос, приходя на сервер, первым делом проходит проверку аутен-

тификации, который, в случае успеха, перенаправляет его на контроллер.

Контроллер обращается к соответствующему сервису, в котором выполняется бизнес-логика запроса, такая как регистрация нового пользователя, добавление предмета и т. д.

Во время своей работы, сервису может понадобиться взаимодействие с базами данных как для чтения, так и для записи. Все это происходит через Repository структуры.

3.2.2. Архитектура базы данных

Рассмотрим используемую сервером SQL базу данных. Ее схема представлена ниже на рис. 4.

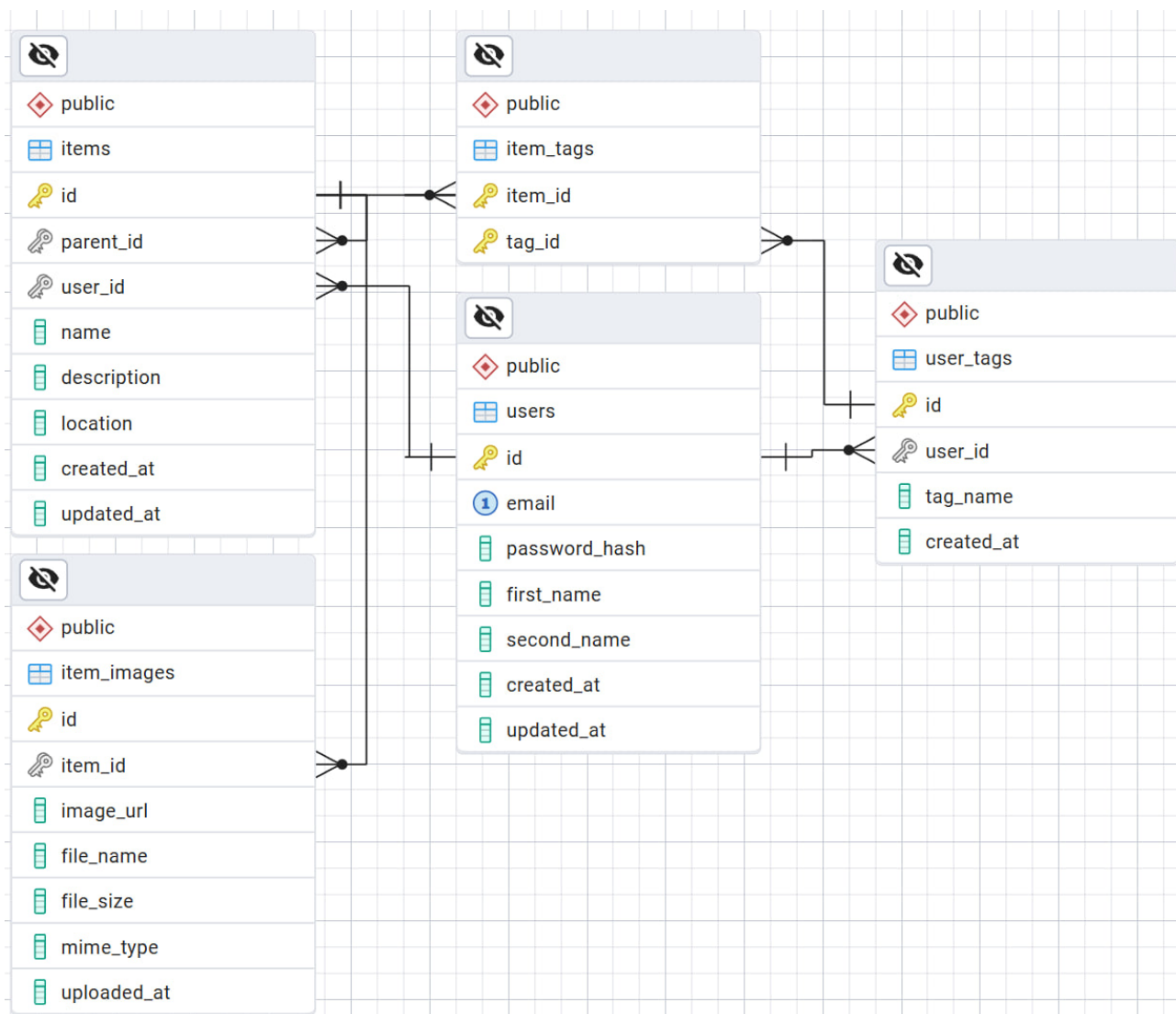


Рис. 4. Схема использующейся базы данных

Она содержит в себе следующие таблицы:

- `users` – пользователи. Содержит информацию о пользователях системы. Каждый пользователь имеет электронную почту, хеш пароля для безопасного хранения, полное имя, а также метки времени создания и последнего обновления учетной записи;
- `items` – предметы. Содержит информацию о личных предметах пользователей. Предметы также могут выступать в роли хранилищ. Каждый предмет принадлежит пользователю, имеет название, описание, местоположение, а также метки времени создания и обновления. Также содержат информацию о тегах через отношение «многие ко многим» с `item_tags`;
- `item_images` – изображения предметов. Содержит информацию о фотографиях и изображениях, привязанных к предметам. Каждое изображение относится к конкретному предмету, содержит URL файла, имя файла, путь к файлу, размер, MIME-тип и метку времени загрузки;
- `user_tags` – теги пользователя. У каждого пользователя могут быть свои собственные теги. Эти теги относятся к предметам через промежуточную таблицу `item_tags`;

В проекте используется Redis для кеширования часто запрашиваемых данных и ускорения работы приложения. В Redis кешируются списки предметов пользователя с их основными характеристиками (название, местоположение, теги), чтобы быстро отображать их без запросов к основной базе данных. Также кешируются результаты поиска по тегам и названиям, так как эти операции выполняются часто. В Redis хранится информация, показывался ли пользователю демонстрационный просмотр предметов (при первом входе в приложение). Дополнительно кешируются иерархии вложенных предметов, так как их рекурсивный запрос из PostgreSQL может быть ресурсоемким.

3.3. Взаимодействие по REST API

Отдельно стоит обсудить то, каким образом построено взаимодействие между клиентом и сервером.

Мы используем REST API. Это популярный подход к проектированию интерфейса взаимодействия, при котором происходит обмен сообщениями без сохранения состояния. В этом случае каждое сообщение, отправляемое на сервер, является самодостаточным и не требует хранить контекст предыдущих сообщений.

Данный подход дает множество преимуществ, среди которых можно выделить простоту реализации, возможность кэширования ответов, а также гибкость и масштабируемость самой системы, реализующей REST API.

Наш REST API использует четыре HTTP метода: GET для получения каких-либо данных, POST – для создания новых, PUT – для изменения существующих и DELETE – для удаления данных.

HTTP запрос может содержать в себе различную информацию, такую как:

- Заголовки, содержащие различную информацию, такую как версия протокола, токен авторизации и т. д.;
- Параметры, которые могут быть параметрами пути или параметрами запроса. В первом случае параметр передается непосредственно в URI, во втором возможна передача сразу множества параметров после символа «?», разделенных символами «&»;
- Тело, в нашем случае являющееся JSON строкой определенного в API формата;

HTTP ответ также состоит из нескольких частей:

- Заголовки, позволяющие клиентскому приложению правильно обработать ответ;
- Тело, которое также содержит JSON строку, определенным образом интерпретируемую клиентом;

- Код – это числовое значение, которое дает понять клиенту, каким образом был обработан запрос. Существует множество кодов ответа на самые разные случаи. Коды 1xx являются информационными, 2xx возвращаются при успешном выполнении запроса, 3xx могут означать неопределенность ответа, либо перенаправлять запрос, 4xx говорят о возникновении каких-либо ошибок на стороне клиента, 5xx – об ошибках на стороне сервера;

Существуют различные способы описать REST API, одним из которых является использование инструмента Swagger, позволяющего в унифицированном виде документировать каждую часть API. В нашем случае была написана спецификация на языке YAML, в которой содержалось описание всех запросов, которые поддерживает сервер.

Данная спецификация может быть запущена в онлайн-редакторах, таких как Swagger Editor [15] для перевода ее в интерактивный вид. Рассмотрим это на примере. Ниже на рис. 5 можно увидеть описание запроса получения детальной информации о предмете/хранилище и о его тегах.

GET
/items/{id}/info
Получение информации о предмете/хранилище

Возвращает детальную информацию о предмете/хранилище и его тегах

Parameters
Try it out

Name	Description
id * required string (path)	id
authCredits * required string (header)	JWT токен аутентификации

Responses

Code	Description	Links
200	Информация о предмете/хранилище Media Type application/json Example Value Schema <pre> { "photo": "JVBORwIKGgoAAANSUHEUgAAAAEAAABCAyAAAFc5JAAADU1EQW42mP6z/C/HgAGwJ/1K3QwAAAB3RUSE-k3ggg==", "name": "Амазон штучатумка", "description": "Красная и дорогая штучатумка", "location": "59.4378:24.7536", "created_at": "string", "updated_at": "string", "father_element": "uuid", "tag": [{ "name": "tag1" }] } </pre>	No links
400	Неверный запрос Media Type application/json Example Value Schema <pre> { "message": "string" } </pre>	No links
403	Доступ запрещен Media Type application/json Example Value Schema <pre> { "message": "string" } </pre>	No links
404	Предмет/хранилище не найдено Media Type application/json Example Value Schema <pre> { "message": "string" } </pre>	No links

Рис. 5. GET запрос получения информации о предмете/хранилище

Мы видим, что запрос требует передачи токена авторизации в заголовке, а также ID в URL запросе.

С помощью кодов ответа, перечень которых также определен в спецификации, клиент может понять, с каким результатом завершился запрос. На каждый код ответа возможна разная реакция – будь то переход на определенную страницу или же вывод сообщения об ошибке для пользователя.

3.4. Код серверной части

Код, принадлежащий серверной части приложения, был написан с использованием интегрированной среды разработки для языка Golang от компании JetBrains. Данная IDE – сильнейшая система, предоставляющая большое количество инструментов для написания кода: искусственный интеллект, автодополнение и исправление, а также множество других возможностей.

На представленном ниже рис. 6 расположена структура кода серверной части приложения.

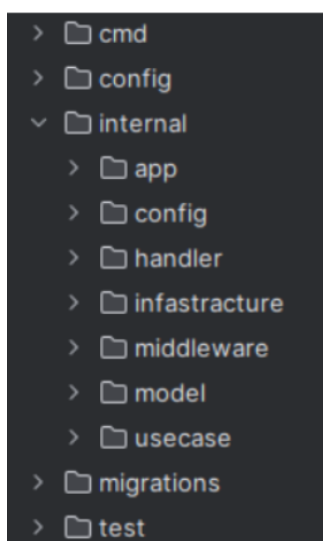


Рис. 6. Структура кода серверной части приложения

Директория с исходным кодом приложения содержит в себе следующие пакеты

- `cmd` – содержит в себе точку входа (`main package`) для запуска приложения;
- `config` – содержит конфигурационный файл для настройки приложения;
- `internal` – содержит основную логику приложения, разделенную на поддиректории:
 - `app` – отвечает за инициализацию и настройку приложения;
 - `config` – включает конфигурации, для внутренних компонентов;

- handler – содержит обработчики HTTP-запросов, DTO (Data Transfer Object) объектов;
- infrastructure – включает реализации внешних взаимодействий (взаимодействия с базами данных и jwt);
- middleware – содержит обработку запросов перед хендлерами;
- model – включает структуры данных, используемые в бизнес-логике;
- usecase – содержит бизнес-логику приложения;
- migrations – включает файлы миграций для базы данных;
- test - пакет реализует интеграционные тесты;

Обработчики HTTP-запросов, бизнес-логика и реализация внешних взаимодействий содержат в своих поддиректориях Unit-тесты.

3.5. Система безопасности

Безопасность приложения обеспечивается за счет использования JWT-токенов в REST API, которые предоставляют надежную аутентификацию и защиту данных.

3.5.1. JWT

В данной части рассмотрим осуществление системы безопасности в приложении. REST API, реализованное сервером, предоставляет два вида запросов – защищенные и незащищенные. Защищенные запросы нуждаются в отправке токена авторизации JWT.

JWT (JSON Web Token) – удобный способ передачи данных авторизации между клиентом и сервером [16]. Пользователь использует незащищенные запросы (представленные у нас запросами регистрации учетной записи и входа через логин и пароль) и получает JSON токен со следующими частями:

- Header – служебная часть. Она содержит информацию о способе обработки токена;

- Payload – полезная нагрузка. Она может содержать различные поля (логин пользователя, его роль и т. д.);
- Signature – подпись, созданная сервером при помощи секретного ключа. Она проверяется на сервере для определения – не был ли выданный

Данная строка должна передавать в заголовке Authorization при обращении к остальным запросам.

3.5.2. Аутентификация

На рис. 7 изображена схема в работе системы аутентификации. Изначально на созданный сервер приходит HTTP-запрос. Его анализом занимается JwtAuthenticationFilter. В начале он проверяет существует ли токен и осуществляется проверка его корректности. Затем из него изымается одно из полей – логин пользователя. JwtAuthenticationFilter передает обращение UserDetailsService. Интерфейс проверяет наличие пользователя в базе данных. В случае отсутствия возможности осуществления проверки – HTTP ошибки 401 (Unauthorized) приходит обратно клиенту.

Процесс валидации токена начинается с его проверки на оригинальность и соответствие сроки через JwtService.

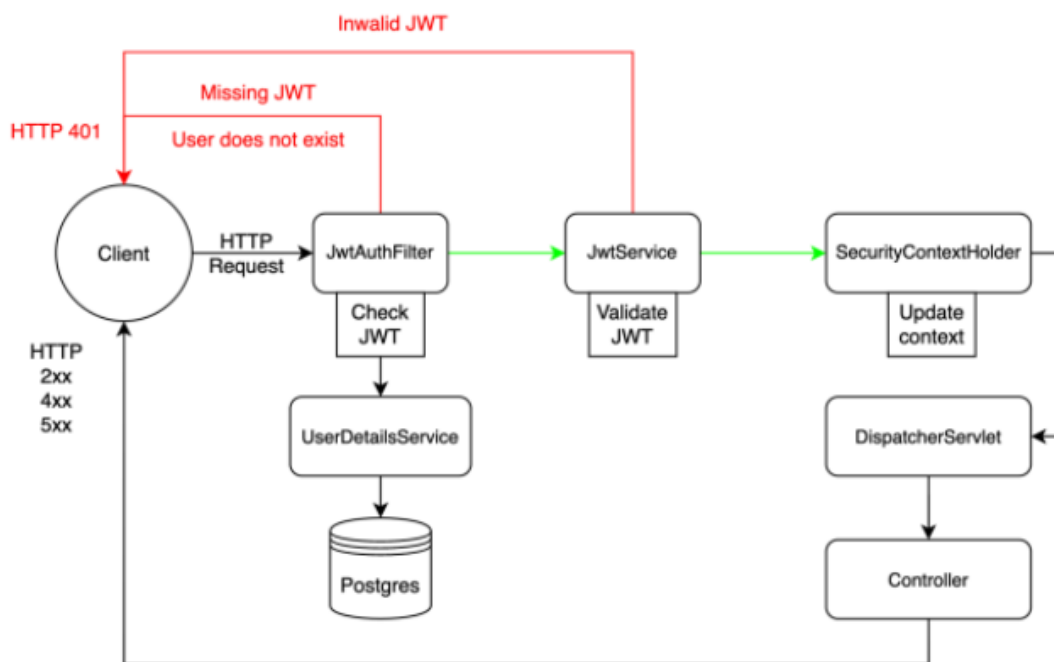


Рис. 7. Схема разработанной системы аутентификации

В случае отсутствия возможности осуществления проверки возвращается ошибка 401.

После успешного завершения всех шагов проверки SecurityContextHolder обновляется с указанием текущего аутентифицированного пользователя, отмечая конец процесса аутентификации.

Затем DispatcherServlet принимает запрос и направляет его соответствующему контроллеру. Следуя логике сервера, он обрабатывает запрос и возвращает клиенту соответствующий HTTP-ответ.

3.6. REST API

3.6.1. Описание конечных точек API

Рис. 8 демонстрирует конечные точки возможных запросов и описание задач, требующих реализации.

Категория	Запрос			
	№	Тип	Наименование	URL
Аутентификация	1	POST	Регистрация пользователя	auth/register
	2	POST	Вход пользователя	/auth/login
	3	POST	Обновление токена	/auth/refresh
Работа с предметами/хранилищами	1	GET	Получить дочерние предметы хранилища с их фото и названием	/items/{id}/content
	2	GET	Получить метаданные предмета/хранилища	/items/{id}/info
	3	PUT	Создать новый предмет/хранилище	/items
	4	GET	Получить все предметы пользователя	/items
	5	PATCH	Обновить информацию о предмете/хранилище	/items/{id}
	6	DELETE	Удалить предмет/хранилище	/items/{id}
Работа с профилем пользователя	1	PUT	Обновить данные профиля	/users/{id}
	2	PATCH	Обновить пароль	/users/{id}/changepass
Работа с тегами	1	GET	Получить теги пользователя	/items/{user_id}/tags
	2	PATCH	Обновить теги предмета	/items/{id}/tags
Поиск	1	GET	По названию, описанию и тегам	/search? name=<id>& tags=<tag>& description=<description>

Рис. 8. Конечные точки серверной части

3.6.2. Структура JSON

Структура JSON – специальный текстовый формат. Он создан для обмена данными между серверной и клиентской частями. Структура устанавливает определенный регламент для заполнения данных, которые передаются в формате ключ-значение и могут вкладываться многократно одной структурой в другую.

Серверная часть приложения и принимает запросы от пользователя, и от-

правляет ему ответ в JSON формате.

ГЛАВА 4. АНАЛИЗ РЕЗУЛЬТАТОВ

В данной главе представлены результаты тестирования серверной части приложения, а также процесс ее развертывания с использованием Docker и облачного сервера. Тестирование включает описание методологии, автоматизацию и отчет о покрытии кода. Анализ подтверждает соответствие системы заявленным требованиям и готовность к эксплуатации в целевой среде.

4.1. Развертывание серверной части

4.1.1. *Dockerfile*

Модульные и интеграционные тесты были созданы для машинной проверки корректности функционала приложения.

Docker [17] – востребованная в индустрии платформа контейнеризации. В данном случае она используется для запаковки серверной части нашего приложения. Более того, именно эта платформа позволяет изолировать приложения в оптимизированные по размеру контейнеры с автоматически развертываемой средой выполнения, обеспечивающие быструю и воспроизводимую деплоймент-миграцию на производственные серверы.

Специально написанный для данной работы Dockerfile отвечает за упаковку серверной части в Docker. Он сообщает определенному инструменту контейнеризации рекомендуемый способ создания образа. Dockerfile представляет из себя набор последовательных инструкций, которые используются для автоматизированного создания Docker-образа:

1. `Golang:1.22` – использование определенной версии языка программирования Go;
2. На этапе подготовки окружения – копирование файлов управления зависимостями (`go.mod`, `go.sum`) с последующей загрузкой требуемых модулей командой `go mod download`, что реализует

- механизм кэширования для оптимизации времени сборки;
3. Создание сборочной директории из всех скопированных исходных файлов и последующее использование `go build` для компиляции главного модуля приложения с указанием выходного бинарного файла по пути `/app_build`;
 4. Создание финального минимального образа на основе `alpine`. В нем располагается рабочая директория `/app_binary`, а затем копируется скомпилированный бинарный файл из промежуточного сборочного образа;
 5. На этапе подготовки к разворачиванию осуществляется перенос файла конфигурации (`config.yaml`) и SQL-миграций (`migrations`) с параллельным выставлением соответствующих прав доступа (0755) на исполняемый файл;
 6. Входящие HTTP-запросы приложение будет принимать с помощью открытия порта 8080. Кроме того, происходит установка точки входа в виде запуска скомпилированного бинарного файла `/app_build`;

4.1.2. Docker-compose

PostgreSQL и Redis – базы данных, чье наличие необходимо в данной работе для удовлетворительного функционирования сервера. Базы данных должны функционировать в разных контейнерах по принципу изолированности. Docker Compose [18] позволяет настроить меж-контейнерное взаимодействие с возможностью последующего запуска всей системы контейнеров одной командой.

Далее представлены части `docker-compose` файла, созданные специально для данной работы:

- Серверное приложение 'backend' контейнеризировано с экспортом

ТСР-порта 8080 на хост-систему, декларативной зависимостью от службы базы данных и автоматизированной оркестрацией запуска;

- Сервис «redis» и «postgres» реализовываются на основе официального образов: Redis и Postgres. В конфигурации указываются параметры подключения (логин, пароль, порт) и выполняется проброс портов контейнеров на соответствующие порты хост-машины;

4.1.3. Сервер

Виртуальная машина с операционной системой Ubuntu была арендована для развертывания серверной части приложения: единственным развернутым на ней портом был порт 80. На данной системе в качестве обратного прокси работал Nginx [19]. Реверсивный промежуточный сервер – один из типов посредников между пользователем и целевым сервером, способный ретранслировать запросы клиентов из внешней сети на сервера, расположенные во внутренней. Главным аспектом его применения является, что клиент видит взаимодействие так, словно запрашиваемые ресурсы располагаются на прокси-сервере.

На данной виртуальной машине для работы Nginx была очищена стандартная конфигурация и создана специальная, перенаправляющая трафик с порта 8080 на порт 80. Он же и был виден из внешней сети.

4.1.4. Автоматизация развертывания

Специальная конфигурация была реализована для автоматизации процесса развертывания Gitlab CD. Запуск процесса инициируется автоматически при обнаружении изменений в главной ветке исходного кода и включает в себя следующие составляющие:

1. В результате работы пайплайна собирается Docker-образ серверного

- приложения и публикуется в Docker Hub;
2. Через `ssh` происходит подключение к удаленной виртуальной машине, где развернута серверная часть;
 3. На сервере выполняется `docker pull`, чтобы загрузить свежий образ из Docker Hub, затем выполняется `docker-compose up -d --force-recreate`, чтобы перезапустить сервисы с новой версией приложения;
 4. В завершение запускается `docker image prune -f`, чтобы удалить неиспользуемые образы и освободить место на сервере;

В итоге, когда код обновляется, в главной ветке репозитория автоматически инициируется последовательность действий. Именно это действие реализует доставку кода до рабочей машины, к которой клиентское приложение отправляет запросы в процессе своей активизации.

На рис. 9. представлено выполнения сценария развертывания с успешным исходом.


```

114 #14 [build_stage 5/6] COPY . .
115 #14 DONE 1.3s
116 #15 [build_stage 6/6] RUN go build -o /app_build ./cmd/server/main.go
117 #15 DONE 20.3s
118 #16 [run_stage 3/4] COPY --from=build_stage /app_build ./app_build
119 #16 DONE 0.8s
120 #17 [run_stage 4/4] RUN chmod +x ./app_build
121 #17 DONE 1.3s
122 #18 exporting to image
123 #18 exporting layers
124 #18 exporting layers 0.5s done
125 #18 writing image sha256:d25b3ddcb4cbabb972794bdde276fe278c21944a77a4162da2f049333d33c09d 0.0s done
126 #18 naming to docker.io/gburan/app_build:latest 0.0s done
127 #18 DONE 0.6s
128 1 warning found (use docker --debug to expand):
129 - JSONArgsRecommended: JSON arguments recommended for ENTRYPOINT to prevent unintended behavior related to
    OS signals (line 14)
130 $ docker push gburan/app_build:latest
131 The push refers to repository [docker.io/gburan/app_build]
132 6c91cea45bb2: Preparing
133 d7cd72428ac4: Preparing
134 6bade105420c: Preparing
135 75654b8eeebd: Preparing
136 75654b8eeebd: Layer already exists
137 6bade105420c: Pushed
138 d7cd72428ac4: Pushed
139 6c91cea45bb2: Pushed
140 latest: digest: sha256:de2f664f3bcae2c00c4203e8ae84e2f5182b6e6397ddcf3634b47062ebcc6dc6 size: 1157
141 $ ssh -o StrictHostKeyChecking=no -i $SSH_PRIVATE_KEY ubuntu@212.233.99.75 'sudo docker stop app || echo 1'
142 Warning: Permanently added '212.233.99.75' (ED25519) to the list of known hosts.
143 app
144 $ ssh -o StrictHostKeyChecking=no -i $SSH_PRIVATE_KEY ubuntu@212.233.99.75 'sudo docker rm app || echo 1'
145 Error response from daemon: No such container: app
146 1
147 $ ssh -o StrictHostKeyChecking=no -i $SSH_PRIVATE_KEY ubuntu@212.233.99.75 'sudo docker pull gburan/app_bui
    ld:latest'
148 latest: Pulling from gburan/app_build
149 da9db072f522: Already exists
150 64dae2213bac: Pulling fs layer
151 03b6e7050b91: Pulling fs layer
152 ea9d085fde06: Pulling fs layer
153 64dae2213bac: Verifying Checksum
154 64dae2213bac: Download complete
155 64dae2213bac: Pull complete
156 ea9d085fde06: Verifying Checksum
157 ea9d085fde06: Download complete
158 03b6e7050b91: Verifying Checksum
159 03b6e7050b91: Download complete
160 03b6e7050b91: Pull complete
161 ea9d085fde06: Pull complete
162 Digest: sha256:de2f664f3bcae2c00c4203e8ae84e2f5182b6e6397ddcf3634b47062ebcc6dc6
163 Status: Downloaded newer image for gburan/app_build:latest
164 docker.io/gburan/app_build:latest
165 $ ssh -o StrictHostKeyChecking=no -i $SSH_PRIVATE_KEY ubuntu@212.233.99.75 'sudo docker run --rm -d -p 808
    0:8080 --name=app gburan/app_build:latest'
166 a908fad9de33e3ece06d6e12c5f80c3521f3eaf9e47cc73bdfa2b408564aa234
167 Cleaning up project directory and file based variables
168 Job succeeded

```

Рис. 9. Успешное выполнение сценария развертывания

4.2. Тестирование серверной части

4.2.1. Используемые технологии

1. Встроенный пакет `testing` – стандартный инструмент для написания автоматизированных тестов в экосистеме Golang. В данной работе он был использован для модульного и интеграционного тестирования серверной части [20]. Более того, этот инструмент позволяет эффективно создавать и выполнять юнит-тесты с минимальной настройкой и интегрируется с большинством редакторов и CI/CD систем;
2. `testify/mock` – фреймворк для создания мок-объектов, использующихся в случае необходимости изоляции тестируемых компонентов от внешних зависимостей. Он предоставляет удобный синтаксис для определения ожидаемых вызовов и поведения функций. Это помогает заменять работу с базой данных внешними API и другими сервисами;
3. `Testcontainers-go` – библиотека для интеграционного тестирования при работе с базой данных PostgreSQL и Redis. Она позволяет запускать сервисы, изолированные от основной операционной системы, в частности базы данных, непосредственно из тестов. Такой подход обеспечивает близкую к реальности среду выполнения без необходимости ручной настройки среды;
4. `Golang-migrate` – инструменты, с помощью которых управляются миграции базы данных в тестах и основном приложении. Он обеспечивает надежное применение SQL-скриптов с верификации;
5. `Httptest` – библиотека, которая является стандартной частью библиотеки Go. С ее помощью осуществляется тестирование REST-контроллеров. Кроме того, она позволяет использовать временные HTTP-серверы и отправлять запросы напрямую к обработчикам, по-

средством чего возможно изолированное тестирование логики API без запуска полноценного веб-сервера;

6. Docker – программное обеспечение, из которого применяются сценарии с поднятием полного окружения для системного (end-to-end) тестирования при необходимости проверки полного цикла обработки запросов, включая сервер и базу данных, с последующим выполнением HTTP-запросов и проверкой ответов. Приведенные тесты запускались в рамках CI-пайплайнов и позволяли убедиться в корректной работе системы;

4.2.2. Описание методологии тестирования

Модульные тесты серверной части приложения проверяют работу небольших отдельных фрагментов кода. При их запуске отсутствует требование поднимать окружение, и их выполнение не занимает большого количества времени.

Проверка корректности работы REST API серверной части заключается в интеграционном тестировании. Перед каждым запуском теста поднимается полноценное серверное окружение, учитывая контейнер Postgres, в котором содержатся тестовые данные.

Тестирование проводится через отправку HTTP запросов и анализ получаемых ответов.

Кроме того, были применены различные техники тест-дизайна, позволяющие минимизировать количество создаваемых тестов. Они использовались в процессе написания тестов и представлены далее:

- Классы эквивалентности – разбиение тест-кейсов на группы с однотипными реакциями системы;
- Граничные условия – тестирование крайних допустимых значений входных данных, где вероятность возникновения ошибок наиболее

высока;

- Попарное тестирование – процесс реализации тестовых случаев для получения удовлетворительной комбинации каждой пары входных параметров, требующийся из-за высокой вероятности допущения ошибки во время их взаимодействия;

4.3. Автоматизация тестирования

Тесты запускались автоматически с помощью системы непрерывной интеграции и доставки GitLab CI. Эта платформа дает возможность гибко конфигурировать pipeline, выполняемый при наступлении определенных событий, таких как создание merge request в главную ветку.

Следующие шаги были выполнены конфигурационным файлом, написанным для модульных и интеграционных тестов серверной части:

1. Клонирование репозитория в систему непрерывной интеграции;
2. Устанавливает необходимую версию языка Go;
3. Загружает все зависимости проекта;
4. Выполняет модульные тесты;
5. Запускает интеграционные тесты, поднимая контейнеры с базами данных;
6. Сохраняет артефакты тестов в виде отчетов и покрытий в систему GitLab CI

Запуск данного сценария происходит автоматически при выполнении следующих условий:

1. Push в ветку master;
2. Merge Request в ветку master;

Результаты пройденного CI можно увидеть на рис. 10

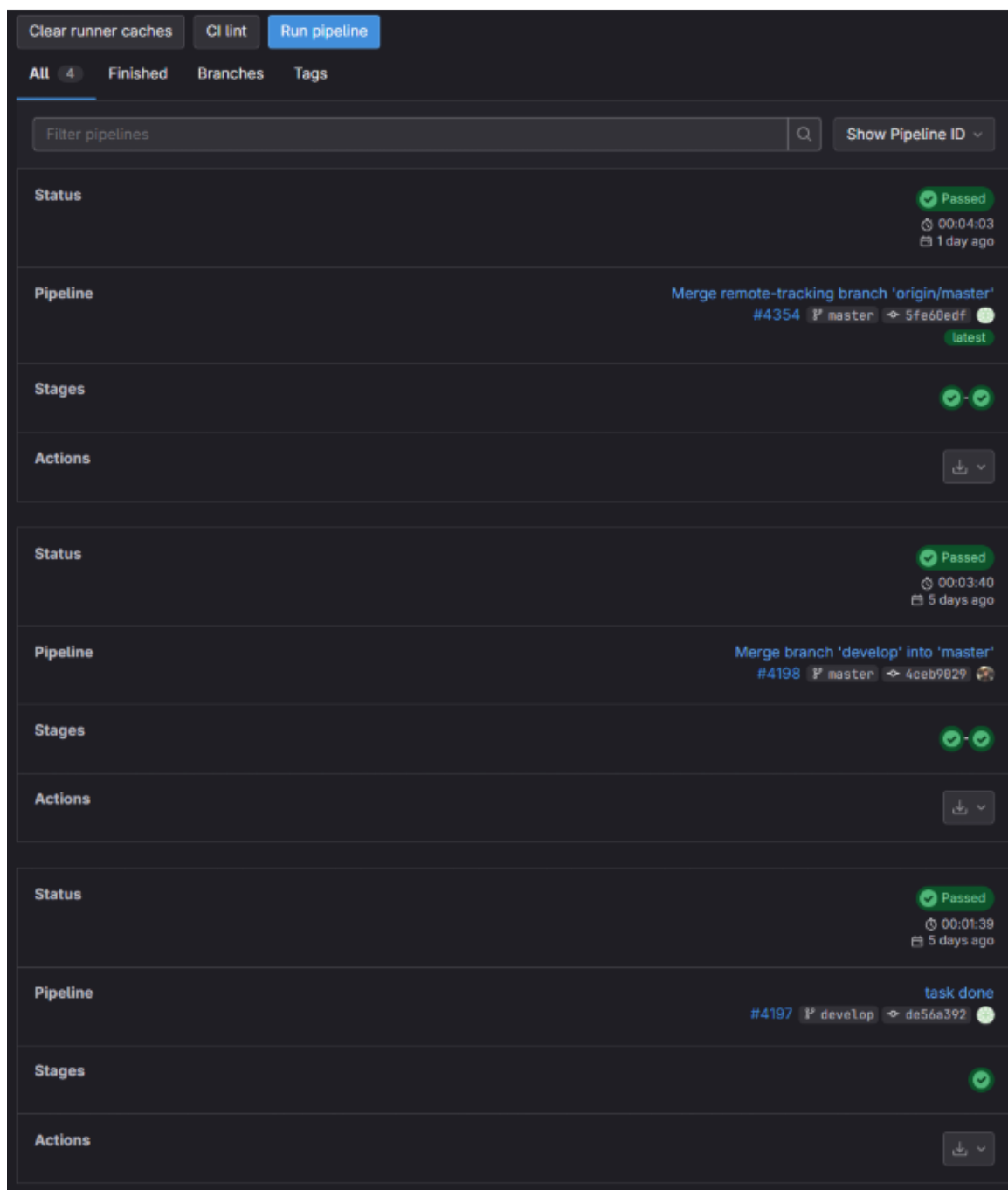


Рис. 10. Успешное выполнение пайплайна

В GitLab так же есть функционал, позволяющий скачать артефакты тестирования.

4.4. Результаты тестирования

4.4.1. Отчет о прохождении тестов

На рис. 11. представлены итоги модульных тестов

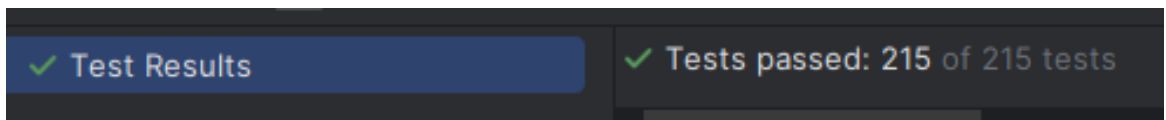


Рис. 11. Модульные тесты

На рис. 12. представлены итоги интеграционных тестов



Рис. 12. Интеграционные тесты

4.4.2. Покрытие кода

Goland Coverage – интегрированный в среду разработки инструмент. В данной работе он использовался для выявления метрики покрытия кода тестами в серверной части.

На рис. 13 продемонстрировано, что 77,5 % строк, принадлежащих серверной части, было покрыто тестами.

✓	service	77.5% files, 73.6% statements
>	cmd	66.7% files, 65.1% statements
✓	internal	40% files, 45.6% statements
>	app	100% files, 100% statements
>	config	100% files, 100% statements
>	handler	40% files, 58.3% statements
>	infastructure	66.7% files, 65.1% statements
>	middleware	100% files, 100% statements
>	usecase	66.7% files, 60% statements

Рис. 13. Кодовое покрытие

ЗАКЛЮЧЕНИЕ

В итоге, была создана серверная часть приложения Android-системы для учета и систематизации предметов. В процессе работы проведен анализ предметной области и сравнение с существующими сервисами. Кроме того, в течение реализации были изучены технические средства и выбраны наиболее подходящие для разработки гибкой архитектуры серверной части. Архитектура данного продукта получилась модульной, основанной на микросервисном подходе. Это дает возможность масштабировать и изменять данное приложение в будущем.

Закончив с реализацией основного функционала, был создан программный интерфейс взаимодействия с серверной частью. Он основан на протоколе HTTP с использованием REST API. Это делает ее универсальной во процессе взаимодействия с ней.

Более того, в работе также есть: ключевые моменты разработки и тестирования кода; изложены основные алгоритмы приложения и работа системы безопасности, контролирующая учетные записи пользователей; настроена непрерывная интеграция и развертывание серверной части приложения, упрощающая обнаружение ошибок и ускоряющая создание приложения за счет стремительных циклов выпуска.

Разработанное приложение в точности соответствует всем поставленным требованиям и является отличным дополнением к существующим ресурсам, посвященным систематизации личных предметов.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [illegible]

15. Swagger Editor [Электронный ресурс] // Swagger: [сайт]. URL: <https://editor.swagger.io/> (дата обращения: 04.Май.2025).
16. Подробно про JWT [Электронный ресурс] // Хабр: [сайт]. URL: <https://habr.com/ru/articles/842056/> (дата обращения: 05.Май.2025).
17. Docker: Accelerated Container Application Development [Электронный ресурс] // Docker: [сайт]. URL: <https://www.docker.com/> (дата обращения: 02.Май.2025).
18. Docker Compose overview [Электронный ресурс] // Docker Docs: [сайт]. URL: <https://docs.docker.com/compose/> (дата обращения: 03.Май.2025).
19. Nginx [Электронный ресурс] // Nginx: [сайт]. URL: <https://nginx.org/ru/> (дата обращения: 01.Май.2025).
20. Основы тестирования Go [Электронный ресурс] // Хабр: [сайт]. URL: <https://habr.com/ru/companies/otus/articles/739468/> (дата обращения: 04.Май.2025).